

Técnicas de Diseño de Algoritmos / Algoritmos y Estructuras de Datos III

Representación de Grafos

2do cuatrimestre de 2025

Departamento de Computación

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Introducción

Matriz de adyacencia

Lista de adyacencia

Grafos implícitos

Introducción

Si queremos escribir un programa sobre grafos
¿Cómo los representamos en memoria?

Depende de nuestro caso de uso:

- ▶ ¿Qué queremos responder?
- ▶ ¿Con qué complejidad?

¿Qué le vamos a pedir a nuestra representación?

- ▶ Inicializar dado un número de vértices y una lista de aristas
- ▶ Preguntar si existe una arista
- ▶ Agregar una arista
- ▶ Eliminar una arista
- ▶ Obtener el vecindario de un nodo
- ▶ Agregar un nodo
- ▶ Eliminar un nodo

Notación

- ▶ $n := |V|$:= cantidad de nodos de G
- ▶ $m := |E|$:= cantidad de aristas de G
- ▶ $N(v)$:= vecindario del nodo v (del inglés *neighborhood*)
- ▶ $d(v) := |N(v)|$ = grado del nodo v (del inglés *degree*)

En general, vamos a asumir que nuestros nodos son los números de 1 a n . Si no lo fueran siempre los podríamos mapear.

Introducción

Matriz de adyacencia

Lista de adyacencia

Grafos implícitos

Definición

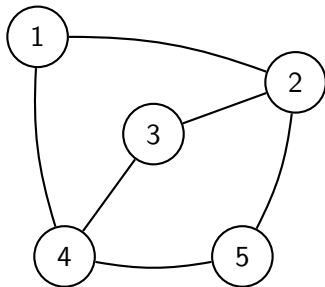
Definición

Vamos a representar el grafo $G = (V, E)$ como una matriz M de $n \times n$.

Si $i, j \in V$, entonces:

- ▶ $M[i][j] = M[j][i] = 0$ si $(i, j) \notin E$
- ▶ $M[i][j] = M[j][i] = 1$ si $(i, j) \in E$

Ejemplo



M	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	0	1
3	0	1	0	1	0
4	1	0	1	0	1
5	0	1	0	1	0

- Complejidad espacial: $\Theta(n^2)$
- Tip: la matriz es simétrica así que podríamos guardar solo una mitad.

Inicializar

```
1: init(cantidad de nodos, lista de aristas):  
2:    $M \leftarrow$  matriz de  $n \times n$  con 0  
3:   for cada arista  $(v, w)$  do  
4:      $M[v][w] \leftarrow 1$   
5:      $M[w][v] \leftarrow 1$   
6:   end for  
7:   return  $M$ 
```

► Complejidad: $\Theta(n^2)$

Aristas

```
1: tieneArista(G, (v,w)):  
2:   return  $G[v][w] == 1$ 
```

```
1: agregarArista(G, (v,w)):  
2:    $G[v][w] \leftarrow 1$   
3:    $G[w][v] \leftarrow 1$   
4:   return G
```

```
1: eliminarArista(G, (v,w)):  
2:    $G[v][w] \leftarrow 0$   
3:    $G[w][v] \leftarrow 0$   
4:   return G
```

► Complejidad: $\Theta(1)$

Vecindario

```
1: vecindario(G, v):  
2:    $res \leftarrow \{\}$   
3:   for  $u \in V$  do  
4:     if  $G[v][u] = 1$  then  
5:        $res \leftarrow res \cup \{u\}$   
6:     end if  
7:   end for  
8:   return  $res$ 
```

► Complejidad: $\Theta(n)$

Introducción

Matriz de adyacencia

Lista de adyacencia

Grafos implícitos

Lista de adyacencia

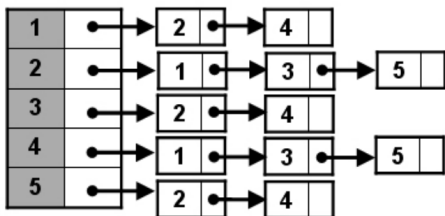
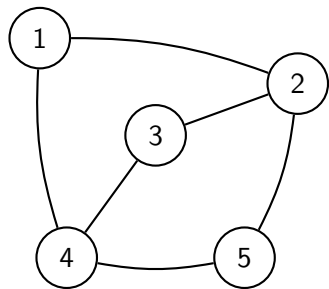
Definición

Vamos a representar el grafo $G = (V, E)$ como un vector donde en la posición correspondiente a cada nodo vamos a almacenar un puntero a su conjunto de vecinos.

Si $v, w \in V$, entonces:

- ▶ $w \in N[v]$ y $v \in N[w]$ si $(v, w) \in E$
- ▶ $w \notin N[v]$ y $v \notin N[w]$ si $(v, w) \notin E$

Ejemplo



► Complejidad espacial: $\Theta(n + m)$

Inicializar

```
1: init(cantidad de nodos, lista de aristas):  
2:    $N \leftarrow$  arreglo de largo  $n$   
3:   for  $v \in 1 \dots n$  do  
4:      $L \leftarrow$  lista vacia  
5:      $N[v] \leftarrow$  puntero a  $L$   
6:   end for  
7:   for  $(v, w) \in E$  do  
8:     A la lista en  $N[v]$  le agregamos  $w$   
9:     A la lista en  $N[w]$  le agregamos  $v$   
10:  end for  
11:  return  $N$ 
```

► Complejidad: $\Theta(n + m)$

Aristas

```
1: tieneArista(G, (v,w)):  
2:   return pertenece( $G[v]$ , w)
```

```
1: agregarArista(G, (v,w)):  
2:   agregar( $G[v]$ , w)  
3:   agregar( $G[w]$ , v)  
4:   return G
```

```
1: eliminarArista(G, (v,w)):  
2:   eliminar( $G[v]$ , w)  
3:   eliminar( $G[w]$ , v)  
4:   return G
```

- ▶ Complejidad pertenece: $O(d(v)) \in O(n)$
- ▶ Complejidad agregarArista: $O(1)$
- ▶ Complejidad eliminarArista: $O(d(v)) \in O(n)$

Vecindario

```
1: vecindario(G, v):  
2:   return G[v]
```

► Complejidad: $\Theta(1)$ ¹

¹Si devuelvo una referencia, ¿Qué ocurre si quiero devolver una copia?

Para pensar

¿Qué complejidad tendrían las operaciones si representáramos los conjuntos de adyacencia con algo distinto de listas?

Analizar representando los conjuntos como:

- ▶ Tablas de hash
- ▶ Vectores
- ▶ AVL
- ▶ ABB
- ▶ Árboles de Fibonacci

Introducción

Matriz de adyacencia

Lista de adyacencia

Grafos implícitos

Grafos implícitos

Se pueden pensar problemas con teoría de grafos sin representarlos explícitamente en memoria.

Mientras podamos definir el conjunto de nodos y, dado alguno de ellos, podamos definir su vecindad, anda.

Por ejemplo:

- ▶ En un tablero o alguna otra estructura regular
- ▶ En una búsqueda por un espacio de soluciones tipo backtracking